HealthTrack Backend Security Implementation and Performance Testing

Todd Simonson

MSCS 699 Computer Science Capstone

Michael Solomon

November 8, 2026

HealthTrack Backend Security Implementation and Performance Testing

The Phase 3 backend implements the Phase 2 design as a FastAPI service over SQLAlchemy models with Alembic migrations. All data is synthetic.

Security Measures

Authentication. Passwords are stored only as bcrypt hashes at cost 12, and an unknown email triggers a dummy hash check so response time does not reveal which accounts exist. Access tokens are JSON Web Tokens that expire after 15 minutes, with the algorithm, issuer, and audience pinned. Refresh tokens are 256-bit random values stored only as SHA-256 hashes and rotated on every use. Presenting a rotated token is treated as theft and revokes every session. Access tokens are tied to a session row, so logout or deactivation ends access at once. Five failed logins lock an account for 15 minutes, a rate limiter returns 429 on repeated attempts, and multifactor accounts must supply a time-based one-time password.

Authorization and auditing. Every patient-scoped route resolves the patient and checks both role and relationship: patients reach only their own record, clinicians and physicians reach only patients on their current care team, administrators see demographics but no readings, and IT administrators see no patient data. An out-of-scope request returns 404 instead of 403 so the API never confirms that another clinician's patient exists, and the denial is written to the audit log. The append-only audit log, which follows the audit control safeguard of the HIPAA Security Rule (U.S. Department of Health and Human Services, n.d.), records the actor, action, record, and outcome of each read and write, never health values.

Input validation and error handling. Pydantic schemas reject unknown fields, and the database repeats each rule as a CHECK, unique, or foreign key constraint. Physically impossible readings are rejected, while unlikely ones are stored with a suspect flag so they never drive an alert. All errors return RFC 9457 problem details (Nottingham et al., 2023) with a request identifier; unhandled exceptions return a generic 500, and responses are marked no-store. The main gap from the Phase 2 design is that the care-team rule is enforced in the API layer, not yet by PostgreSQL row-level security policies, which are planned for final integration.

Performance Testing

The benchmark (scripts/benchmark.py) migrated a fresh database and loaded 100 synthetic patients with 135,850 vital sign rows. Requests ran in-process through the full stack, excluding network time. The run used macOS 26.5.1 on an Apple silicon Mac mini with 8 cores, Python 3.12.13, and SQLite 3.53.3. The test suite passed 237 of 237 tests on the same machine with 97.8% line coverage, above the 80% target in requirement NFR-16.

Table 1

Endpoint Latency Measured In-Process on the Development Machine

Operation	n	p50 (ms)	p95 (ms)
POST /auth/token (bcrypt cost 12)	10	252.3	283.9
POST vitals, single reading	200	4.3	9.3
POST vitals/batch, 500 readings	20	61.8	83.4
GET /patients (worklist page)	200	2.8	3.4
GET vitals, 24-hour page	200	4.4	5.4
GET vitals, 7 days, 200 per page	200	6.9	22.6
GET vitals/latest	200	6.4	8.7
GET vitals/daily-summary, 21 days	200	12.0	15.1
GET daily-activity/summary	200	2.7	3.8
POST /patients/search	200	3.0	3.9

Note. Times include authentication, authorization, validation, and the audit write; p50 is the median.

As Table 1 shows, every read endpoint stayed under 23 ms at the 95th percentile, well inside the 300 ms target from requirement NFR-08. Batch upload sustained about 7,775 readings per second, and a replayed batch stored nothing, so device retries are safe. Login was the slowest operation at 252 ms, an intended cost of the bcrypt work factor.

The composite index on patient and recorded time had the largest effect. For a 24-hour window on a patient with 10,521 readings, the query returned 2 rows in 0.02 ms with the index. Without it, SQLite read and sorted every reading for the patient, taking 2.98 ms, a 149-fold difference that grows with patient history. These results cover one machine, SQLite, and a single client; concurrent load and PostgreSQL are left for Phase 6.

References

Nottingham, M., Wilde, E., & Dalal, S. (2023). *Problem details for HTTP APIs* (RFC 9457). Internet Engineering Task Force. https://doi.org/10.17487/RFC9457

U.S. Department of Health and Human Services. (n.d.). *Summary of the HIPAA Security Rule*. Retrieved November 8, 2026, from https://www.hhs.gov/hipaa/for-professionals/security/laws-regulations/index.html